HPC-Boost: A Binary driven ML Framework for boosting HPC based Runtime Attack Detection

***Abstract*—Modern microprocessors have special-purpose registers called "Hardware Performance Counters" (HPC), which can be programmed to keep counts of various low-level hardware and software events during a program's execution. Due to its ability to provide critical information about an application's runtime behaviour, HPCs have been widely explored for runtime attack detection. Although a plethora of events can be monitored, it is important to restrict to a few specific events as the number of physical HPC registers is limited and multiplexing too many events on the given registers leads to unreliable HPC data stream at runtime. So, many HPC based attack detection tools usually fix a few events apriori, but without a proper quantitative justification. Moreover, they use the same set of HPC events to monitor different binaries. In this work, we propose a novel "binary features-driven HPC event recommendation framework" to boost runtime attack detection. We use statistical methods to rank the events based on the sampled HPC data streams of the programs, such that events more suitable for anomaly detection are ranked higher. This ranking mechanism does not consider the functioning associated with the events to avoid unwanted bias. Then for a wide variety of binaries, we extract appropriate features and apply a machine learning scheme to train a mapping between these binary features and their HPC event rankings. Finally at the deployment phase, this pipeline of "feature extractor + trained model", is used to get HPC event recommendation for any binary in a lightweight manner. Our experimental evaluations show significant performance improvement on multiple anomaly detectors using the HPC events recommended by our framework. The modularity of the design of our proposed solution makes it easily adaptable for recommending HPC events in other potential use cases.**

***Index Terms*—Hardware Performance Counters, Anomaly Detectors, Runtime Attack Detection, Runtime Monitoring, Binary Analysis, Lightweight Feature Extraction, Machine Learning, Time series data analysis, Perf**

I. Introduction

Modern processors have special purpose registers called "Hardware Performance Counters (HPCs)" which can be programmed to capture counts of various hardware and software events during the execution of an application. Some examples of monitorable HPC events are the number of branch misprediction observed, the number of TLB Flush operations executed, CPU cycles, etc. HPCs are used for various purposes like hardware and software debugging and profiling [7], [8], [9], performance analysis [4], [6], power monitoring [10], [11], [5].

Runtime attacks can cause severe issues, including the program's failed execution and damage to the host system itself [23], [24], [25]. Several runtime intrusion prevention and anomaly detection tools have been developed which monitor some observable runtime properties of the system during the execution of a program and flag an anomaly as soon as they detect significant deflection in those observed properties. The HPC events provide crucial low-level details about an application's runtime execution. They reside at special-purpose HPC registers, which makes it very hard for any intruder to sabotage them. Thus HPC events have been widely explored in providing security against [12] - [18] runtime attacks, especially for the development of several runtime anomaly detection tools.

A. HPC Event Selection: Why it is important

Several tools like perf_events [1] and PAPI [2] exist which provide an easy to use interface to program these HPC registers to capture specific events. Typically, 400-500 HPC events are monitorable in modern processors [13]. But the constraint arises because of the limited number of physical HPC registers present in modern CPUs (usually 4 in Intel [26] and 6 in AMD [27]). As our experimental setup uses an intel processor, the number of HPC events that we can reliably keep track of for any application is almost 4. Suppose we try to monitor more than four events. In that case, these registers are virtualized or multiplexed (time-shared), which makes the count of events more unreliable because of large measurement errors in the event counts [22].

B. Considering Binary's Influence in Event Selection:

Most of the work on HPC based anomaly detectors rarely pay attention to the properties of actual binary under attack. Many people have tried to study the nature of binaries and extract relevant features from them to capture their behavior [28]. We claim that leveraging binary-feature extraction for binary-driven HPC based anomaly detection can open a significant direction of research in enhancing these anomaly detectors using the binary's influence.

C. Solution Framework Outline:

In this work/paper, we propose to fill this gap of HPC-Event selection or recommendation. Our solution framework is environment-specific. Once any hardware+software environment is fixed, we provide a mechanism to rank all the monitorable events in terms of their goodness, where goodness score is defined by their effectiveness for anomaly detection purposes. We use this mechanism to create a labeled dataset of workload mapped to their corresponding event rankings.

Then we extract a representation of each binary in terms of relevant features, and train ML model to learn this mapping between binary feature vectors and the event rankings. This way we develop a solution particular to that given ecosystem which can be later deployed in a lightweight manner at runtime.

The pipeline of the deployed solution is shown in the Figure 1 consisting of the binary feature extractor and the trained ml model, which takes in the binary of an application and

gives an appropriate ranking of all the 400+ monitorable HPC. Then finally the anomaly detector uses the best among these recommended events to monitor the binary at runtime and perform effective runtime attack detection.

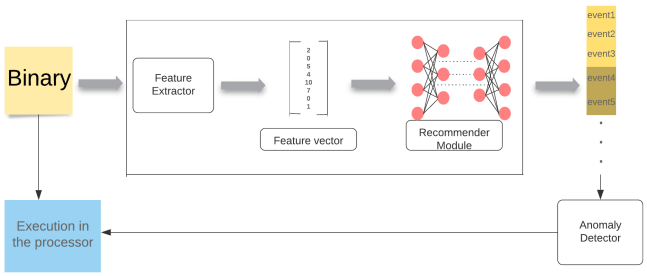


Fig. 1: Flow of our pipeline after deployment for runtime attack detection.

So our key contributions are:

- We give a quantitative justification as to why a set of few particular events make more sense to be monitored than others, for anomaly detection, and how this choice varies with different target binaries.
- Our modular and generalizable solution design considers all 400+ monitorable events irrespective of what they signify, to minimizes user's bias.

II. Related Work and Motivation

Previously, a considerable amount of effort has gone into event selection. To highlight a few directions of approach, author's have tried to pick events based on their intuitive understanding of what would work best for their use case. For instance, [17] had shortlisted the total number of instructions retired, branch instructions retired, cache stores, completed I/O operations, and floating-point operations. Some authors [15], [21] have viewed event data streams as features to a subsequent machine learning model, and then applied different feature elimination and ranking techniques on them to reduce the number of features to the ones with the most suspected predictive power. Few others like [14] have tried statistical methods to capture their desired criteria. [3], [19] also indicate for multiple other works which HPC events they have chosen along with their purpose of selection. But most of them suffer from inadequate rigor in formalising their event selection procedure and a lot of them lack key details. [21] also talks about how microarchitectural events can be in general inefficient in reliably capturing the nature of the program. But what we care about in this paper is among the huge pool of available events to monitor, is it possible to find a set of events which are more relevant compared to others for a particular use case like anomaly detection in programs in our case. Furthermore, we try to establish whether this process can be made binary-driven; effectively removing the human in the loop, which is something yet untouched.

Our claims are:

- Different binaries should be monitored with different sets of HPC events for better anomaly detection.
- For anomaly detection, all possible monitorable HPC events should be treated equally without considering their actual meaning. Instead of selecting HPC events by rough intuition, we should do a formal investigation of the observed HPC stream to select relevant HPC events for monitoring.

We tested the effectiveness of the HPC event ranking done by our framework on several popular anomaly detection models. We found that there was a significant difference in performance when doing anomaly detection using top events in our ranking versus bottom events in our ranking. We have observed that as we go down in the ranking, the effectiveness of anomaly detection reduces. We also observed that anomaly detectors perform better when using different events for different binaries. Also, right now we have done only binary-driven event recommendations. This can be further extended in future to a full end-to-end solution design where each component of the anomaly detector is also tuned on the basis of the influence from the target binary.

III. HPC-BOOST Framework

A. Overall Pinepine summary

What we essentially begin with is a 'function', that will map a binary to an appropriate ranking of the monitorable events in the given environment. This function is what we address in our **"Ranking Module"**, in which we come up with a ranking mechanism for these available events. Then we need a **Recommender Module**, which is basically an ML model that learns this 'function'. And since a model needs some distinguishable representation of the target binary to be given as input, we need the **Feature Extractor**. Thus our HPC event recommendation system mainly consists of 3 components:

- **Feature Extractor**: Binary Feature Extraction Module.
- **Ranking Module**: HPC Event ranking module.
- **Recommender Module**: DL model which learns the mapping between Binary Feature to HPC event Ranking module.

For each binary, our ***"Ranking module"*** monitors every HPC event across 5 different independent executions. So finally it has number of datafiles=5*number of binaries*number of events. Then it applies statistical tests for time-series analysis of the stored data streams to score and rank events based on their "goodness" for anomaly detection purpose. Note that the cumbersome nature of the process undertaken by our Ranking Module is evident. So it is impractical to run this heavy process at deployment phase. Hence our ranking mechanism is just used to generate the training dataset.

Our ***"Feature Extractor module"*** extracts relevant binary features in the form of a vector. The goal of the Feature Extractor is to capture the essence of the binary in a light-weight manner.

Then in training time, the model in our ***"recommender Module"*** trains on these binary's feature Vectors and Probability scores corresponding to the HPC event ranking for that binary.

At testing time, our ***"feature extraction module"*** extracts the feature vector from the new binary, and our trained ***"recom-***

mender module" obtains the HPC event ranking for this feature vector. Note that this pipeline won't incur much overhead in-field as both these modules are not computationally heavy once deployed. Then the recommended events are fed to the anomaly detector ahead, for runtime monitoring.

B. Ranking Module (Event scores for goodness)

We use various tests to give scores to the HPC event data-streams captured by running training benchmarks. The goal of this model is to rank events in terms:

- Consistency across multiple runs (to reduce false positives).
- Capture a more holistic view of the system activities.
- Overall predictability, so that they can be reliably captured by the parameters of the model later in the anomaly detection phase.
- Sensitivity towards malicious behaviour. (to reduce false negatives).

So with respect to the stream of HPC data for a particular binary, we assign the following scores to each of the moniterable event: 1) Spread Score 2) Trend Score 3) Correlation Score and, 4) Stationarity Score. Higher scores for a test indicates that event is better suited to be monitored for that aspect. And most of the tests that we have performed are non-parametric because even though parametric tests perform better when the data is coming from the assumed underlying distribution with reasonable guarantee, we cannot ascertain such a distribution to all the HPC event data and in those cases non-parametric tests are more powerful(Hirsch et al., 1991).

***Spread Score*:** We simply take Spread Score = reciprocal of interquartile range = $\frac{1}{Q_3 - Q_1}$, where $Q_3 = 75^{th}$ percentile mark and $Q_1 = 25^{th}$ percentile mark of the data. We do this because this score will be outlier resilient as outliers don't significantly perturb the quartiles. This is done so that, when a malicious execution will cause a fluctuation in the target event, these fluctuations should not get obscured in the inherent jitters in the of the event data stream. And for the malware

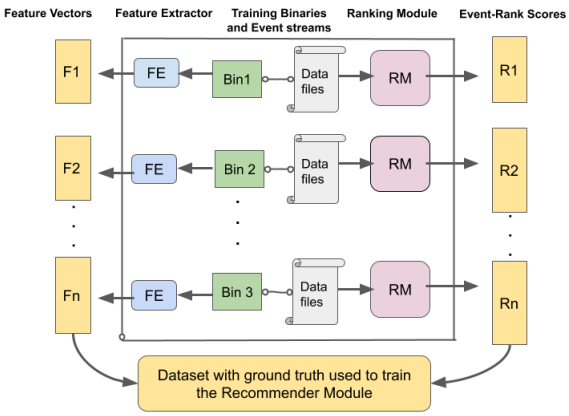
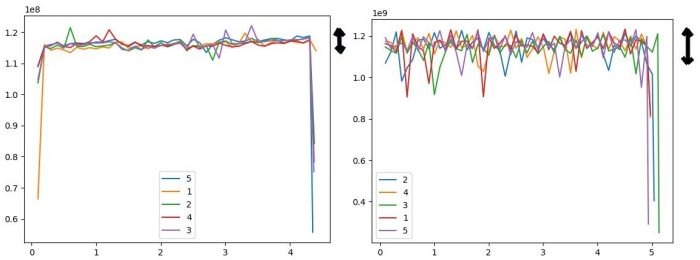


Fig. 2: The training data for Recommender Module consists of these Feature vectors F and Probability scores R as labels corresponding to their event Rankings.



(a) "Branches" event (b) "Instructions" event

Fig. 3: Black arrow on top left indicates the spread of event counts time series, over 5 runs. The x-axis: time step in minutes and y-axis: event counts

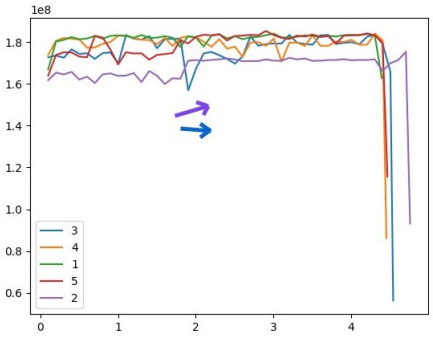


Fig. 4: Plot of event counts for memory-stores event. x-axis: time step in minutes and y-axis: event counts. The purple and blue arrows indicate the overall trend of the data in purple run has a slight upward trend compared to the blue run, owing to the dip in blue in between. All these together will decrease the trend score of this event.

to actually achieve anything significant, these fluctuations will have to persist just long enough to cease being outliers and actually cause a change in the spread of the data. We consider the average of this quantity over multiple runs as the final event score. This test however independent of the time series nature of the data though. See Figure 3.

***Trend Score*:** For this we have done the Mann Kendall Trend Test, a non-parametric trend test which is especially powerful in detecting a trend when there actually exists one, compared to other popular tests like t-test [33]. We use this test statistic value to evaluate the consistency of the time series data over multiple runs as shown in Figure 4. The closer the trends across multiple runs, the higher the score.

***Correlation Score*:** Now to gain more holistic view of the system, we try to observe a bunch of events that have less correlation among them. For this we calculate:

$$\forall event_i,\ t_{ij} = Correlation(event_i, event_j)$$
$$Score(event_i) = \min(\textstyle\sum_{\forall j:j\neq i}[1 - \mid t_{ij} \mid], \frac{1}{\sum_{\forall j:j\neq i}|t_{ij}|})$$

This idea is also shown in [14], but they have used Pearson Correlation. In our work we have used its non-parametric counterpart instead, which is Spearman Correlation. Since Spearman

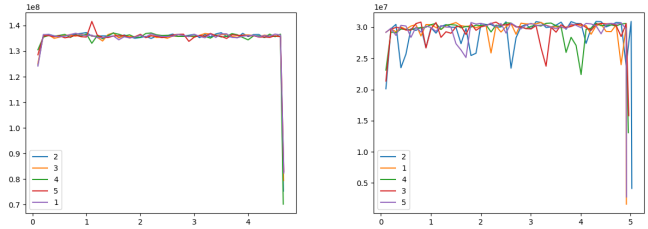
branch mispredicts	0.1397
mem loads	0.1396
l2 hits	0.1396
branches retired	0.1395

(a) Top4 events

flt pt arith ops	1.145e-08
cpu cycles	1.0009e-08
itlb flush	0.203e-08
offcore rqsts data rd	0.27e-09

(b) Bottom4 events

TABLE I: Scores of the Ranked Events for Egg-Dropping Puzzle benchmark



(a) "Branch Mispredicts" event (b) "Offcore data rd rqsts" event

Fig. 5: Comparison of the plots of top ranked and bottom ranked event, over 5 runs. The x-axis: time step in minutes and y-axis: event counts

Correlation uses ordinal values to correlate, it can also evaluate a general monotonic but non-linear relationship between the variables, unlike Pearson Correlation which evaluates only the linear relationships.

***Stationarity Score*:** Another important aspect of time series analysis is stationarity, which gives us a measure of how unwavering are the statistical properties of that time series data over time. The more stationary the data is, the more reliably the anomaly detector will be able to capture it because many models rely on it. Depending on the length of the time series data at hand, we classified the data streams as short(25-100) and long100 and beyond). For the short ones, even though non-parametric unit root tests like PP [34] test, are available; its asymptotic theory basis makes it insufficient standalone. Hence a combination of that with the KPSS [35] test of stationarity is better suited, as recommended by [36] also. And for the long ones, we go with the well established ADF [37] test and also combine it with PP test as the error in the time series value cannot be assumed to be entirely homoscedastic. But all these mentioned tests suffer from sensitivity to structural breaks. And since we expect to have structural breaks near the start and end of the series due to possible artifacts near the start and end of the execution of a program; we also use Zivot-Andrews [38] test in tandem with these to allow for at most 2 structural breaks in the series and hence ensure a sufficiently robust stationarity test score overall in the end.

Seasonality is modelled in the form of an appropriately fitted sinusoidal beforehand and removed for better analysis of stationarity tests and trend tests. Using this seasonality further in the form of separate criteria to assign meaningful scores has been left for later explorations. Each of the score categories is normalized by the max event score, in the end, to prevent arbitrarily large values in one score category from overshadowing other score categories when we combine them to obtain a final score for each event. And this combination is just plain addition for now but can be made more sophisticated by doing weighted addition depending on the importance of different scores in a particular application.

They [21] have used PCA to judge the predictive power of the events which can be an inadequate metric to use. Moreover, what we have shown is that though attack specific scores can be designed further to assess the goodness of events, even a filter that is constructed based on rules as simple and intuitive as the ones mentioned in the beginning, can be significantly effective in selecting relevant events from a huge pool, to optimize detection of something like integrity violations.

C. *Feature Extractor Module (Binary Feature extraction)*

In this module, we extract relevant features from the binaries into a feature vector such that the vector represents that binary and sufficiently distinguishes it from others in context of anomaly detection. We want that the feature vector captures these behavioral aspects of the binary adequately enough. This module is used both at training and testing phases. We extract feature vectors from binaries statically in a lightweight manner without executing the binaries.

We disassemble and analyze binary using Angr [29] tool to extract CFG (control flow graph) of the binary. We extract feature vectors considering these two important aspect of binary:

- Approximation of the number of times a type of instruction would be committed at runtime.
- The sequence of instructions that would be observed at runtime.

We extract these three kinds of features from binaries:

1) **CFG features:** No. of basic blocks, instructions in each basic block, number of loops, etc.
2) **Approximate runtime throughput (hits) of different of Instructions in the binary:** This feature was extracted from CFG of the binary. We traversed the CFG using BFS and then maintained the count of different instructions in a basic block, these instructions were further segregated based on the operands of the instructions (whether operands were registers or memory addresses). First we use dataflow analysis techniques to calculate the number iterations for as many loops as possible. Note that it is impossible to statically find the number of iterations of all loops. Thus if we are not able to statically prove the number of iterations of a loop then we use a random number (upper bounded by 500) as its number of iterations. While adding the counts of each instruction, we multiply the count with number of loop iterations for all basic blocks that are part of that loop.
3) **Ngram features:** This feature helps us model the sequence of instructions [28]. We read the binary file of the application as a text file and extract n-grams [30] [31]. Since this vector can get really big for larger binaries we also applied a feature cleaning step in which we remove low count n-grams.

D. Recommender Module (ML model for binary driven event ranking)

Architecture:

The model is a simple neural network made using 3 Linear Layers, each layer followed by ReLU activation. It takes the feature vector extracted from binaries and then maps them to scores (normalized to be less than 1) of events provided by the Ranking Module. This neural network learn the mapping between binary feature vector and HPC event scores.

Loss Functions used: L2-norm and Optimizer used: Adam Optimization

Training details and methodology:

To see how our framework performs on a diverse set of data, we take a mix of standard dataset from MiBench [32] and self constructed programs of a variety of popular algorithms; and run them on different large input sizes for getting reasonable runtime to monitor HPC events streams. We split this data into 20-80 train-test sets and further use 5 fold cross validation on the training set for tuning the hyperparameters.

Our Feature Extractor provides features from the binaries and the Ranking Module helps us obtain the target probability scores for events. While running the binaries to get event data streams to feed them into the ranking module, we restart the system everytime a single run of a particular binary gets over to ensure authenticity of the HPC data.

IV. Experiments

A. Setup

For hardware-software ecosystem for our experiments, we fix intel i5-3337U @1.8GHz x 4 core, Ivybridge; with 8 GB of RAM. We use 64-bit OS, Ubuntu 18.04.4 LTS. We use Perf [1], which is a popular utility in the Linux Kernel, which is used by applications to capture suitable events through these HPC registers and read them at desired frequencies and granularity. The version of Perf in our kernel was 4.14.18.

Most of the HPC papers ignore software events provided by perf_events utility, but we consider all the events irrespective of their behaviour. We test and present our framework's performance boost on three standard anomaly detection models: One Class SVM, K-nearest neighbors, and Isolation Forest (which is a Random Forest based anomaly detector). [20], [21].

To simulate runtime attacks, we modify some instruction opcodes, operands in the code section of the binary. We also insert function calls and branch/jmp instructions at various places in the ELF file of the binary. We train anomaly detectors on HPC streams of the good version of the benchmarks and see if it flags anomaly after runtime attack.

B. Results and inferences

1) Trend of FP and FN values as we go down in the event ranking: For each anomaly detector, the trend of Fp and Fn (as we slide down the event ranking with a window size of 4 events), averaged over all test binaries is shown in in Figure 6 and Figure 7 respectively.

We can see that the plots have a slight increasing trend which confirms that as we go down in the event ranking given the performance of anomaly detector degrades.

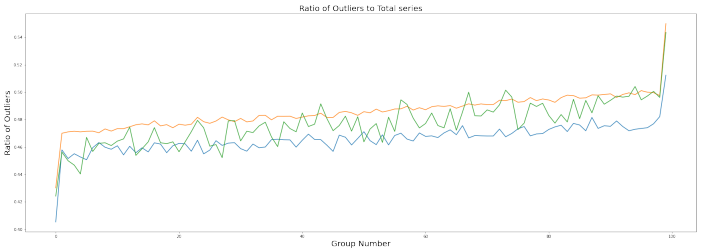


Fig. 6: Sliding window plots for false positives.

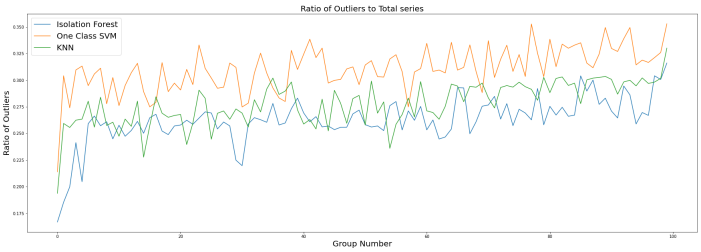


Fig. 7: Sliding window plots for false negatives.

2) Binary driven event selection matters: Here we swap top 4 (and bottom 4) events of two test binaries and study performance of anomaly detection. From Table II We observed that monitoring binary 1 with top 4 events of binary 2 and vice versa degrades performance of anomaly detection. Thus binary specific event selection is important.

2*Anomaly Detector	Case for evaluation			
	Binary 1 with Events for Binary 1	Binary 2 with Events for Binary 2	Binary 1 with Events for Binary 2	Binary 2 with Events for Binary 1
Isolation Forest	0.1571	0.1334	0.1974	0.1968
KNN	0.1372	0.1607	0.1999	0.1932
One Class SVM	0.1738	0.1643	0.2137	0.2018

TABLE II: Results for experiment where we swap events for binaries

3) Average False Positives and False Negatives for different models: Table 3 and Table 4 show performance of anomaly detection for top 4 and bottom 4 events in our ranking respectively. This shows using top 4 events is significantly better as compared to bottom 4 events for anomaly detection. This validates that our framework ranks HPC events according to their effectiveness for anomaly detection.

	Isolation Forest				KNN				One Class SVM			
Binary Name	TP	TN	FP	FN	TP	TN	FP	FN	TP	TN	FP	FN
box_stacking_problem	0.7368	0.9873	0.2632	0.0127	0.9654	0.8392	0.0346	0.1608	0.9421	0.7929	0.0579	0.2071
convex_hull	0.5306	0.6889	0.4694	0.3111	0.9388	0.8667	0.0612	0.1333	0.9592	0.8131	0.0408	0.1869
ford_fulk	0.8696	0.5946	0.1304	0.4054	0.9565	0.7432	0.0434	0.2568	0.9855	0.786	0.0145	0.214
freivald_algorithm	0.7884	0.9713	0.2116	0.0287	0.7692	0.9375	0.2308	0.0625	0.9807	0.8472	0.0192	0.1528
Average FN and FP	0.8358	0.7958	0.1642	0.2042	0.9402	0.8237	0.0597	0.1763	0.9169	0.7942	0.0831	0.2058

TABLE III: TP, TN, FP, FN for top 4 HPC events.

	Isolation Forest				KNN				One Class SVM			
Binary Name	TP	TN	FP	FN	TP	TN	FP	FN	TP	TN	FP	FN
box_stacking_problem	0.6492	0.9842	0.3508	0.0158	0.98	0.8393	0.02	0.1607	0.8246	0.8929	0.1754	0.1071
convex_hull	0.7143	0.6333	0.2857	0.3667	0.9388	0.8667	0.0612	0.1333	0.898	0.9333	0.102	0.0667
ford_fulk	0.8986	0.9324	0.1014	0.0676	0.9566	0.7432	0.0434	0.2568	0.942	0.7162	0.058	0.2838
freivald_algorithm	0.7885	0.9942	0.2115	0.0058	0.7692	0.9375	0.2308	0.0625	0.9423	0.9688	0.0577	0.0312
Average FN and FP	0.791	0.757	0.209	0.243	0.9402	0.6796	0.0598	0.3204	0.9104	0.6021	0.0896	0.3979

TABLE IV: TP, TN, FP, FN for bottom 4 HPC events..

V. Conclusions and Future Work

Selection of appropriate HPC events is crucial for HPC based runtime attack detection. Also the same set of events may not be effective for anomaly detection of all workloads or binaries. In this work we presented a novel binary driven HPC event recommendation framework for boosting the performance of HPC based runtime anomaly detection. We demonstrated the performance boost of our framework on multiple anomaly detectors.

The modularity of the design of our framework makes it possible to modify any module of our pipeline independently, without disturbing other components. For example the ranking module can be used to score and rank HPC events based on their goodness for other use cases of HPCs. We can also modify the Feature Extractor module to accept other entities like hardware IP/module to make appropriate recommendations of HPCs for their deployment. We plan to investigate such extensions of our framework to recommend HPC events like power monitoring [10], [11], [5] and performance analysis [4], [6]. Another possible direction of exploration could be an attempt to develop novel extensions in anomaly detection models such that their parameters or architecture can gain benefit from features extracted out of binaries.

References

[1] Weaver, V. M. Linux perf event features and overhead in International Workshop on Performance Analysis of Workload Optimized Systems (2013).

[2] Mucci, P. J., Browne, S., Deane, C. Ho, G. PAPI: A portable interface to hardware performance counters in Department of Defense HPCMP Users Group Conference (1999), 7–10.

[3] Foreman, J. C., "A Survey of Cyber Security Countermeasures Using Hardware Performance Counters", arXiv e-prints, 2018.

[4] Max Plauth, Christoph Sterz, Felix Eberhardt, Frank Feinbube, Andreas Polze: Assessing NUMA Performance Based on Hardware Event Counters. IPDPS Workshops 2017: 904-913.

[5] Utilizing Hardware Performance Counters to Model and Optimize the Energy and Performance of Large Scale Scientific Applications on Power-Aware Supercomputers. IPDPS Workshops 2016.

[6] Karl Fürlinger, Daniel Terpstra, Haihang You, Philip Mucci, Shirley Moore: Enabling Data Structure Oriented Performance Analysis with Hardware Performance Counter Support. Euro-Par Workshops 2008.

[7] Brian J. N. Wylie, Bernd Mohr, Felix Wolf: Holistic Hardware Counter Performance Analysis of Parallel Programs. PARCO 2005:

[8] Alexandre Kandalintsev, Renato Lo Cigno, Dzmitry Kliazovich, Pascal Bouvry: Profiling cloud applications with hardware performance counters. ICOIN 2014.

[9] Enric Gibert, Raúl Martínez, Carlos Madriles, Josep M. Codina: Profiling Support for Runtime Managed Code: Next Generation Performance Monitoring Units. IEEE Comput. Archit. Lett. 14(1): 62-65 (2015).

[10] Xin Liu, Li Shen, Cheng Qian, Zhiying Wang: Dynamic Power Estimation with Hardware Performance Counters Support on Multi-core Platform. ACA 2014.

[11] Younghyun Kim, Sangyoung Park, Youngjin Cho, Naehyuck Chang: System-Level Online Power Estimation Using an On-Chip Bus Performance Monitoring Unit. IEEE Trans. on CAD of Integrated Circuits and Systems 30(11).

[12] Congmiao Li, Jean-Luc Gaudiot: Detecting Malicious Attacks Exploiting Hardware Vulnerabilities Using Performance Counters. COMPSAC (1) 2019: 588-597.

[13] B. Singh, D. Evtyushkin, J. Elwell, R. Riley, and I. Cervesato, "On the detection of kernel-level rootkits using hardware performance counters," in Proceedings of the ACM Asia Conference on Computer and Communications Security, pp. 483–493, 2017

[14] R. Elnaggar, K. Chakrabarty and M. B. Tahoori, "Run-time hardware trojan detection using performance counters," 2017 IEEE International Test Conference (ITC)

[15] Vinayaka Jyothi, Xueyang Wang, Sateesh Addepalli, Ramesh Karri: BRAIN: BehavioR Based Adaptive Intrusion Detection in Networks: Using Hardware Performance Counters to Detect DDoS Attacks. VLSI Design 2016: 587-588.

[16] Xueyang Wang, Jerry Backer: SIGDROP: Signature-based ROP Detection using Hardware Performance Counters. CoRR abs/1609.02667.

[17] Corey Malone, Mohamed Zahran, Ramesh Karri: Are hardware performance counters a cost effective way for integrity checking of programs. STC@CCS 2011.

[18] Prashanth Krishnamurthy, Ramesh Karri, Farshad Khorrami: Anomaly Detection in Real-Time Multi-Threaded Processes Using Hardware Performance Counters. IEEE Trans. Inf. Forensics Secur. 15: 666-680 (2020).

[19] Kanad Basu, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri: A Theoretical Study of Hardware Performance Counters-Based Malware Detection. IEEE Trans. Inf. Forensics Secur. 15: 512-525 (2020).

[20] Sanjeev Das, Jan Werner, Manos Antonakakis, Michalis Polychronakis, Fabian Monrose: SoK: The Challenges, Pitfalls, and Perils of Using Hardware Performance Counters for Security. IEEE Symposium on Security and Privacy 2019: 20-38.

[21] Boyou Zhou, Anmol Gupta, Rasoul Jahanshahi, Manuel Egele, Ajay Joshi: Hardware Performance Counters Can Detect Malware: Myth or Fact? AsiaCCS 2018: 457-468.

[22] Yirong Lv, Bin Sun, Qingyi Luo, Jing Wang, Zhibin Yu, Xuehai Qian: CounterMiner: Mining Big Performance Data from Hardware Counters. MICRO 2018: 613-626.

[23] Oren, Y., Kemerlis, V. P., Sethumadhavan, S., and Keromytis, A. D., "The Spy in the Sandbox – Practical Cache Attacks in Javascript", arXiv e-prints, 2015.

[24] Luka Malisa, Kari Kostiainen, Thomas Knell, David M. Sommer, Srdjan Capkun: Hacking in the Blind: (Almost) Invisible Runtime User Interface Attacks. IACR Cryptol. ePrint Arch. 2017: 584 (2017)

[25] Love Kumar Sah, Sheikh Ariful Islam, Srinivas Katkoori: An Efficient Hardware-Oriented Runtime Approach for Stack-based Software Buffer Overflow Attacks. AsianHOST 2018: 1-6.

[26] Intel Itanium Architecture Software Developer's Manual. Intel Corporation, 2010.

[27] BIOS and Kernel Developer's Guide (BKDG) for AMD Family 15h Models 10h-1Fh Processors. Advanced Micro Devices, Inc., 2015

[28] H. Xue, S. Sun, G. Venkataramani and T. Lan, "Machine Learning-Based Analysis of Program Binaries: A Comprehensive Study," in IEEE Access, vol. 7, pp. 65889-65912, 2019. doi: 10.1109/ACCESS.2019.2917668.

[29] F. Wang and Y. Shoshitaishvili, "Angr - The Next Generation of Binary Analysis," 2017 IEEE Cybersecurity Development (SecDev), Cambridge, MA, 2017, pp. 8-9, doi: 10.1109/SecDev.2017.14.

[30] Raff, E., Zak, R., Cox, R. et al. An investigation of byte n-gram features for malware classification. J Comput Virol Hack Tech 14, 1–20 (2018). https://doi.org/10.1007/s11416-016-0283-1.

[31] M. Hassen, M. M. Carvalho and P. K. Chan, "Malware classification using static analysis based features," 2017 IEEE Symposium Series on Computational Intelligence (SSCI), Honolulu, HI, 2017, pp. 1-7, doi: 10.1109/SSCI.2017.8285426.

[32] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538), Austin, TX, USA, 2001, pp. 3-14, doi: 10.1109/WWC.2001.990739.

[33] Önöz, B. Bayazit, M.. (2003). The Power of Statistical Tests for Trend Detection. Turkish Journal of Engineering and Environmental Sciences. 27. 247-251.

[34] Peter C. B. Phillips, Perron, P. (1988). Testing for a Unit Root in Time Series Regression. Biometrika, 75(2), 335-346. doi:10.2307/2336182.

[35] Giraitis, L., Leipus, R., Philippe, A. (2006). A Test for Stationarity versus Trends and Unit Roots for a Wide Class of Dependent Errors. Econometric Theory, 22(6), 989-1029. Retrieved September 21, 2020, from http://www.jstor.org/stable/4093211.

[36] Arltová, Markéta Fedorová, Darina. (2016). Selection of Unit Root Test on the Basis of Length of the Time Series and Value of AR(1) Parameter. Statistika - Statistics and Economics Journal. 96. 47-64.

[37] Xiao, Z., Phillips, P. (1998). An ADF coefficient test for a unit root in ARMA models of unknown order with empirical applications to the US economy. The Econometrics Journal, 1(2), 27-43. Retrieved September 21, 2020, from http://www.jstor.org/stable/23116930.

[38] Zivot, E., Donald W. K. Andrews. (1992). Further Evidence on the Great Crash, the Oil-Price Shock, and the Unit-Root Hypothesis. Journal of Business Economic Statistics, 10(3), 251-270. doi:10.2307/1391541.